Plateforme automatisee de detection de vulnerabilites web

SAST - DAST - SCA + mecanisme personnalise + metriques + CI/CD

Abderrahmen ABDELJALIL

Ecole Polytechnique Sousse - Securite des Applications Web & Email

Juin/Novembre 2026

Contexte & objectifs

Concevoir une solution reproductible qui automatise la detection et l evaluation des vulnerabilites

- Objectif : combiner SAST (code statique), DAST (exécution) et SCA (dependances) dans une meme plateforme
- Evaluer la capacite reelle de chaque outil a detecter les vulnerabilites PRESENTES (ground truth issue des TP1-TP4)
- Centraliser les resultats dans un format unique, automatiser via CI/CD (GitHub Actions) et produire des rapports exploitables
- Apport personnel : un mecanisme de detection complementaire (regles Semgrep + authz_checker BOLA/BFLA)

Architecture de la plateforme

Un depot Git unique, un deploiement, un scan, un rapport

- Cibles (Docker) : Juice Shop v20.2.0 (:3000) - crAPI (:8888) - Roundcube 1.6.21 (:8080) + SonarQube (:9000)
- Scanners : Semgrep (regles OWASP + regles custom) - SonarQube - OWASP ZAP (Automation Framework) - Trivy - npm audit
- Orchestration Python/Bash : run_sast / run_dast / run_sca / run_all -> results/<date>/ (archives horodatees)
- Normalisation : normalize.py convertit 7 outils dans un schema JSON unique -> comparaison possible
- Evaluation : metrics.py confronte les findings au ground truth -> TP / FN / Recall / Precision
- CI/CD : GitHub Actions a chaque push -> SAST + SCA + artefacts (90 j) + branche results

Ground truth : le referentiel d evaluation

11 vulnerabilites - methodologie : tests manuels sur l instance deployee + TP valides

ID	Cible	Vulnerabilite	Gravite	Statut
JS-001	juice-shop	Injection SQL sur POST /rest/user/login (champ email	critical	confirmed
JS-002	juice-shop	XSS réfléchi via la barre de recherche produit	high	candidate
JS-003	juice-shop	XSS stocké via les avis produit	high	candidate
JS-004	juice-shop	En-têtes de sécurité HTTP manquants (CSP, Referrer-P	low	confirmed
CR-001	crapl	BOLA : accès au véhicule d'un autre utilisateur (IDO	high	confirmed
CR-002	crapl	BFLA : endpoint administratif accessible avec un jet	high	confirmed
CR-003	crapl	Manipulation de statut de commande (remboursement sa	medium	pending
CR-004	crapl	Secrets hardcodés dans la configuration (JWT_SECRET=	medium	confirmed
RC-001	roundcube	Divulgation de versions serveur (Server: Apache/2.4.	low	confirmed
RC-002	roundcube	Dépendances applicatives obsolètes / CVE connues (co	high	pending
JS-005	juice-shop	Dependances npm vulnerables (package-lock / advisory	high	pending

Ex. JS-001 : injection SQL sur /rest/user/login confirmee par notre test (payload " OR 1=1-- -> JWT admin@juice-sh.op obtenu sans mot de passe).

SAST : Semgrep + SonarQube + regles custom

Analyse statique sur les 3 sources clonées

- Semgrep (profils p/owasp-top-ten) : 241 alertes sur les 3 cibles - rapide, oriente vulnerabilites
- SonarQube (conteneur, API d export) : 667 issues - oriente qualite, mais RATE la SQLi (recall 50%)
- Regles custom (personnelles, taint mode) : SQLi template-literal (TP2), command injection (mini-lab), XSS DOM (TP3), secrets hardcodes (CR-004)
- Validation sur fixtures : 4 alertes sur le fichier vulnerable, 0 sur le fichier corrige (non-regression)
- Preuve : la regle custom detecte JS-001 sur routes/login.ts:34 la ou SonarQube ne voit rien

DAST : OWASP ZAP (Automation Framework)

Scan dynamique contre les applications en fonctionnement

- Baseline (spider passif) sur les 3 cibles puis scan actif complet (spider + activeScan, budgets de temps)
- Rapports HTML + JSON (templates risk-confidence-html / traditional-json) archives par date
- Resultats : en-têtes de sécurité manquants, divulgation de versions (Server / X-Powered-By), cookies sans SameSite
- Limite observee : ZAP ne detecte PAS la SQLi en JSON sur le login non authentifie (recall 66.7% sur le GT)
- ZAP ne couvre pas les failles d autorisation (BOLA/BFLA) -> complement oblige (voir slide 9)

SCA : Trivy + npm audit

Dependances applicatives et configuration

- Trivy image : 70 CVE (Juice Shop) / 149 (postgres crAPI) / 194 (Roundcube) en HIGH+CRITICAL
- Trivy fs : sources + secrets + misconfigurations -> 591 findings au total (dont 9 secrets, 153 misconfigs)
- npm audit (Juice Shop) : 55 paquets vulnerables (7 critical, 22 high) - lockfile regenere automatiquement
- Detection des secrets hardcodes de crAPI (JWT_SECRET=crapi, mots de passe base) -> corrobore CR-004

Mecanisme complementaire personnalise

La valeur ajoutee du projet : detecter ce que les outils standards ratent

- 1) Regles Semgrep maison (scanners/semgrep/rules/) : 4 regles, mode taint, metadata CWE/OWASP/GT
- 2) authz_checker.py (Python) : reproduction automatisee de la methodologie du TP4 -
- 2 comptes (foulen/foulana) + vehicules seedes + requetes croisees token/ressource
- BOLA : GET /vehicle/<uuid victime>/location avec le jeton attaquant -> HTTP 200 + PII du proprietaire
- BFLA : GET /workshop/api/management/users/all + DELETE /admin/videos/<id> avec un jeton USER -> pas de 401/403
- Aucun outil standard (ZAP/Semgrep/SonarQube) ne detecte ces failles : complementarite prouvee

Resultats : metriques vs ground truth

Recall = vulnerabilites du GT detectees ; Precision = findings rattaches (GT non exhaustif)*

Outil	TP	FN	Recall	Findings	Precision*
Semgrep	2	0	100.0%	241	0.8%
Regles custom	2	0	100.0%	12	16.7%
Trivy	3	0	100.0%	591	0.3%
npm audit	1	0	100.0%	55	1.8%
authz_checker	2	1	66.7%	4	50.0%
OWASP ZAP	2	1	66.7%	26	7.7%
SonarQube	1	1	50.0%	667	0.1%

Conclusion : aucun outil n est exhaustif -> la fusion normalisee des 7 sources couvre l integralite du ground truth.

* Precision indicative a relecture manuelle : le GT ne reference pas toutes les vulnerabilites reelles (findings non classes).

Exploitation : scenario avant / apres

TP2 Phase 5 - remediatio SQLi (prepared statements) sur routes/login.ts

Mesure	Avant (vulnerable)	Apres (corrige)
Alertes totales (Semgrep)	52	49
Alertes sur routes/login.ts	3	0
Alertes introduites par le fix	-	0

- Correctif TP2 : requetes preparees (parametres lies) au lieu du template literal
- Les 3 alertes SQLi (express-sequelize-injection + 2 regles custom) disparaissent, 0 regression
- Usage CI : chaque push re-verifie automatiquement que la vulnerabilite reste corrigee

Automatisation CI/CD - GitHub Actions

github.com/ChahbaniRayen/app-securite-

- Declencheurs : push/PR sur les chemins critiques (scanners, custom, orchestrator, ground-truth) + planning hebdo + manuel
- Jobs : SAST (clone des 3 sources + Semgrep + SonarQube optionnel) | SCA (Trivy + npm audit) | DAST (ZAP baseline a la demande)
- Archivage : artifacts 90 jours + push automatique sur la branche results (historique horodaté des scans)
- Pipeline local equivalent : ./orchestrator/run_all.sh (meme chaine en une commande)
- Statut : SAST vert en CI ; SCA corrige (tag d action) ; rapports PDF regenerables a tout moment

Conclusion & perspectives

- Solution reproductible : deploy.sh + run_all.sh + report_pdf.py = toute la chaine en 3 commandes
- Evaluation objective : ground truth fonde sur les TP, metriques TP/FN/Recall/Precision par outil
- Complementarite prouvee : Semgrep/Trivy 100% recall | SonarQube 50% | ZAP 66.7% | authz_checker seul sur BOLA/BFLA
- Exploitation : scénario avant/apres qui prouve la validation automatique des corrections
- Perspectives : connecter crAPI a ZAP via OpenAPI, enrichir le GT (CR-003), dashboard HTML, seuils de qualite en CI